

Exercício 1 – Trabalhando com as palavras reservadas *static* e *final*.

Objetivo do exercício – Neste exercício modificaremos a classe **Banco** para implementar o *design pattern* **Singleton**.

Dica: Duplicque a versão do projeto desenvolvido na lista de exercício anterior (“**Exercício 3 – Pesquisando clientes pelo nome e sobrenome**”) para que você possa manter o histórico de evolução do código do projeto Banco durante as listas de exercícios. Outra possibilidade é usar um sistema de controle de versões. O projeto duplicado possui uma classe chamada `TesteBancoExercicio3` que poderá ser removida.

Tarefas:

O nosso projeto do banco necessita de uma hierarquia mais completa de pacotes porque iremos criar uma classe chamada **RelatorioClientes** que pertence ao pacote **banco.relatorios**.

1. Mude a declaração de pacotes em cada classe para refletir o diagrama apresenta na Figura 1.

Modifique a classe **Banco** para implementar o *design pattern* **Singleton**

1. Modifique a classe **Banco** para criar um **método público estático**, chamado **getBanco()**, que retorna a instância da classe **Banco**
2. A instância deve ser um **atributo estático privado**. Da mesma maneira, torne o **construtor privado**.

Atualize a classe **ContaCorrente** conforme diagrama.

1. Implemente os métodos para retornar e atribuir respectivamente o valor do cheque especial.

Modifique a classe **RelatorioClientes**

No exercício anterior do nosso projeto bancário, o relatório do clientes estava dentro do **método main** da classe **TesteBanco**. Neste exercício, movemos este código para a classe **RelatorioClientes** no pacote **banco.relatorios**. Sua tarefa será modificar esta classe, fornecida com este enunciado, para utilizar o objeto *Singleton* **Banco**.

1. Dica: Maiores detalhes sobre o que deve ser feito no exercício estão presentes no código fornecido da classe `RelatorioClientes` a partir de código marcado com blocos de comentários `/**`
`*/`

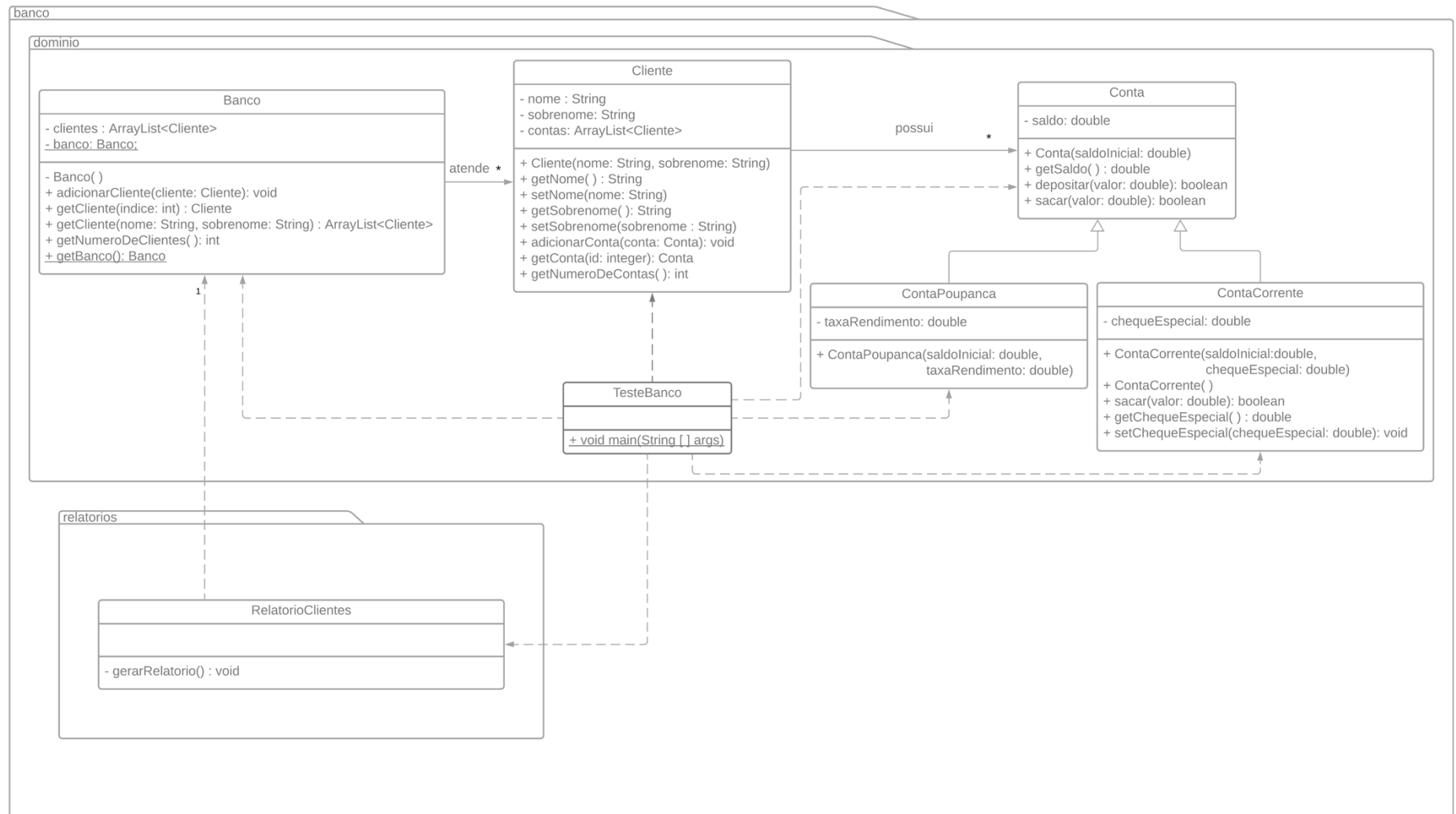


Figura 1 – Diagrama de classes com nova estrutura de pacotes para o projeto Banco, modificação da classe Banco para se tornar um *Singleton* e inclusão da classe RelatorioClientes.

Modifique a classe `TesteBanco`:

1. Implemente o código necessário na classe **TesteBanco**, também fornecida com este enunciado.
Dica: Maiores detalhes sobre o que deve ser feito no exercício estão presentes no código fornecido da classe **TesteBanco** a partir de código marcado com blocos de comentários `/** **/`
2. Execute o programa **TesteBanco** (Lembre-se sempre de depurar todo código fornecido por mim para verificar o entendimento do mesmo). A seguinte saída deverá ser exibida:

```
Criando uma conta corrente para o cliente Bruno Henrique.
Criando uma conta corrente para o cliente Everton Ribeiro
Criando uma conta corrente para o cliente Filipe Luis.
Criando uma conta corrente para o cliente Gabriel Barbosa.
Criando uma conta corrente para o cliente Diego Alves.
Diego Alves autorizou o cadastro de sua conta corrente como sendo conjunta com a cliente Lorena Lara.

RELATÓRIO DE CLIENTES
=====

Cliente: Bruno, Henrique
    Conta Poupanca: Saldo atual de R$ 50.000,00.
    Conta Corrente: Saldo atual de R$ 220.000,00.
    Cheque especial de R$ 40.000,00

Cliente: Everton, Ribeiro
    Conta Corrente: Saldo atual de R$ 45.000,00.
    Cheque especial de R$ 30.000,00

Cliente: Filipe, Luis
    Conta Corrente: Saldo atual de R$ 70.000,00.
    Cheque especial de R$ 0,00
    Conta Poupanca: Saldo atual de R$ 220.000,00.

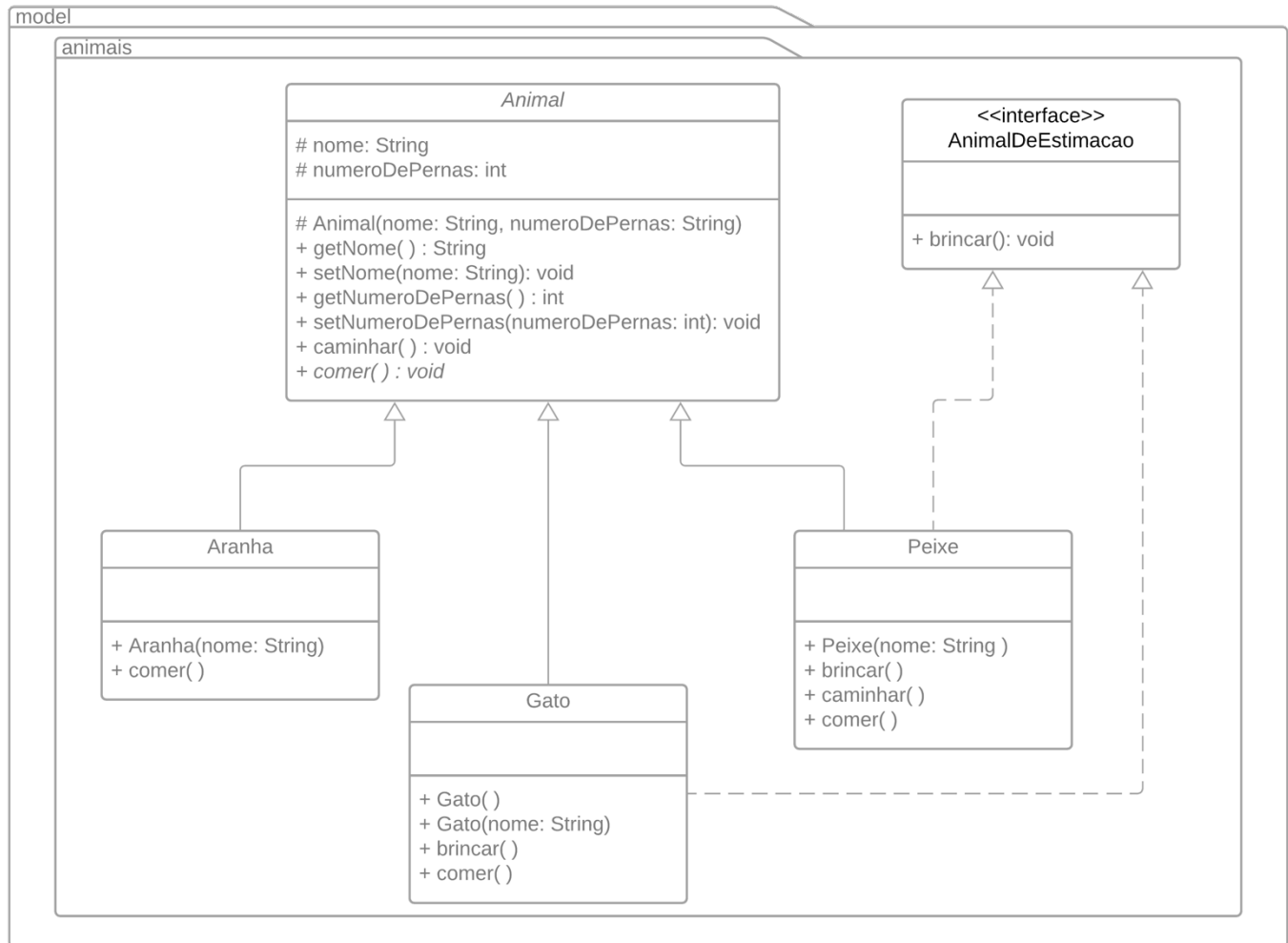
Cliente: Gabriel, Barbosa

Cliente: Diego, Alves
    Conta Corrente: Saldo atual de R$ 50.000,00.
    Cheque especial de R$ 0,00

Cliente: Lorena, Lara
    Conta Corrente: Saldo atual de R$ 50.000,00.
    Cheque especial de R$ 0,00
```

Exercício 2– Trabalhando com Interfaces e Classes Abstratas

Objetivo do exercício – Neste exercício, criaremos uma hierarquia de animais cujo ancestral em comum é a classe abstrata **Animal**. Várias classes do tipo **Animal** implementarão a interface chamada **AnimalDeEstimacao**. Treinaremos com variações destes animais, seus métodos e polimorfismo.



Tarefas:

1. Crie a **classe Animal**, que é a **superclasse abstrata** de todos os animais.
 - a. Declare um atributo inteiro **protected** com o nome **numeroDePernas**, que armazena o número de pernas do animal.
 - b. Declare um atributo **protected** do tipo **String** representando o nome do animal.
 - c. Defina um **construtor protected** que inicia o atributo **numeroDePernas**.
 - d. Declare o **método abstrato comer**.
 - e. Declare o **método concreto caminhar** que exibe uma mensagem do tipo “<nome do animal> anda com <numero> pernas.”
 - f. Crie **métodos get/set** conforme diagrama.

2. Crie a **classe Aranha**.
 - a. A classe **Aranha** herda da **classe Animal**
 - b. Defina um construtor que chama o construtor da superclasse para especificar que todas aranhas possuem 8 pernas.
 - c. Implemente o **método comer**
3. Crie a **interface AnimalDeEstimacao** especificada no **diagrama UML**
4. Crie **classe Gato** que herda de **Animal** e implementa **AnimalDeEstimacao**.
 - a. Defina um construtor que recebe um **parâmetro do tipo String** que especifica o nome do gato. Este construtor deve chamar o construtor de sua superclasse para especificar que todos gatos possuem **4 pernas**.
 - b. Defina outro construtor que não recebe parâmetros. Dentro deste construtor chame o construtor anterior (utilizando a palavra **reservada this**) e passe uma **string vazia** como argumento.
 - c. Implemente o método da **interface AnimalDeEstimacao**.
 - d. Sobrescreva o **método comer** para exibir no *console* que **gato** come ração.
5. Crie a **classe Peixe** que herda de **Animal conforme diagrama de classes**. Sobrescreva os métodos de **Animal** para especificar que **peixe não anda e não possui pernas**.

Treine com estas Classes

6. Crie um programa chamado **TesteAnimais**. Dentro do **método main**, crie e manipule instâncias das classes criadas acima.

Inicie com:

```
Peixe p = new Peixe("Nemo");
Gato g = new Gato("Tom");
Animal a = new Peixe("Peixoto");
Animal ab = new Aranha("Aracnilda");
AnimalDeEstimacao ae = new Gato("Pudim");
```

Experimente:

- a) Chamar métodos em cada objeto.
- b) Fazer “*casting*” de objetos (conversões).
- c) Utilizar polimorfismo.

Uma possível saída:

O peixe Nemo possui 0 pernas

Nemo não anda, nada.

Nemo brinca com anzol.

Nemo come minhoca

O gato Tom possui 4 pernas

Tom anda com 4 pernas.

Tom brinca com com a bola de meia.

Tom come ração.

O animal Peixoto possui 0 pernas

Peixoto não anda, nada.

Peixoto brinca com anzol.

Peixoto come minhoca

O animal Aracnilda possui 8 pernas

Aracnilda anda com 8 pernas.

Aracnilda come insetos

Pudim brinca com com a bola de meia.

O animal de estimação Pudim possui 4 pernas

Pudim anda com 4 pernas.

Pudim come ração.